
Design and Implementation of a RISC-V Processor Architecture: A VGA Labyrinth Game on FPGA

Lucia Pecora
Student ID: 237780
Course: Advanced Logic Design
Academic year: 2025/2026

Contents

1	Introduction	3
2	System architecture	4
2.1	Block Diagram	4
2.2	Datapath diagram	5
2.3	Instruction Format	6
3	Modules description	7
3.1	Arithmetic Logic Unit (ALU)	7
3.2	Register File	8
3.3	Control Unit	9
3.4	ALU Control	10
3.5	Immediate Extender	11
3.6	Program Counter	12
3.7	Comparator	12
3.8	Datapath	13
3.9	Instruction Memory	14
3.10	Data Memory	15
4	Processor Top-level	17
4.1	Interface	17
4.2	Instantiated components	17
4.3	Benefits of the separation of responsibilities	18
5	Functional verification	19
5.1	Core Modules Verification	19
5.2	Datapath verification	21
5.3	CPU verification	22
6	Hardware peripherals	23
6.1	Controller SPI	23
6.2	Controller VGA	24
6.3	GPIO & Seven-segment display	26
7	System integration	28
7.1	Memory-mapped I/O	28
7.2	Address Decoder & Map Memory	28
7.3	Integrated system audit	29
8	Game software	30
8.1	Firmware Architecture and Game Logic	30
8.2	Compilation, Boot, and Hardware Initialization Flow	31
8.3	System Simulation and Pre-Implementation Validation	32
9	FPGA implementation	34
9.1	Physical Constraints and Top-Level Adaptations	34
9.2	Resolution of Architectural Issues	34
9.3	Timing Analysis and Derived Clock	34
9.4	Bitstream Generation and Hardware Testing	35

10 The Game: Design and Final Testing	37
10.1 The game elements	37
10.2 The level design	37
10.3 Gaming on the hardware	38
11 Conclusions	39

Chapter 1

Introduction

The project consists of designing and implementing a complete System on a Chip on a Digilent Nexys 4 DDR FPGA. The core of the system is a custom processor based on the single cycle RISC-V RV32I architecture, which runs the bare metal firmware for an interactive video game.

The game is a maze where the user must guide a ball from the starting point to the finish line, avoiding the holes. The movement is physically controlled by tilting the board, using data from the integrated three axis accelerometer (ADXL362). The visual rendering happens in real time on an external monitor via a VGA interface.

First, we designed the hardware. This phase included the datapath, the control unit, the CPU (ALU, Register File, Program Counter), and the integration of memory blocks (BRAM). We also developed the controllers for the external peripherals (an SPI serial interface for the accelerometer, a VGA sync generator, and a GPIO module for switches, LEDs, and seven segment displays), which operate through a memory mapped bus.

Then, we focused on the software. We wrote the video game firmware in C (including a boot routine in Assembly). This code is optimized to calculate the ball's physics and resolve collisions using logical and arithmetic shift operations.

Finally, the entire synthesis, simulation, and programming process was managed using the Xilinx Vivado development environment.

Chapter 2

System architecture

Designing a digital system requires defining its high-level architecture. This chapter illustrates the structural organization of the project using block diagrams for both the complete system and the CPU datapath. This approach helps to map the interfaces, data flows, and interactions between the different hardware modules. The chapter presents also the instruction formats of the RISC-V RV32I ISA (Instruction Set Architecture) supported by the processor. Analyzing these formats is essential to understand the decoding logic that controls the Control Unit and the operation of the Immediate Extender.

2.1 Block Diagram

The system is organized into three main levels:

- **RISC-V Core:** the datapath of the single cycle processor, responsible for fetching, decoding, and executing the program instructions;
- **Memory Subsystem:** composed of the ROM (which stores the firmware to run) and the RAM (which provides a workspace for temporary data, the stack, and the image pixels to display on the screen);
- **I/O Peripherals:** dedicated controllers (SPI, VGA, and GPIO) that act as an interface between the processor logic and the physical devices outside the FPGA.

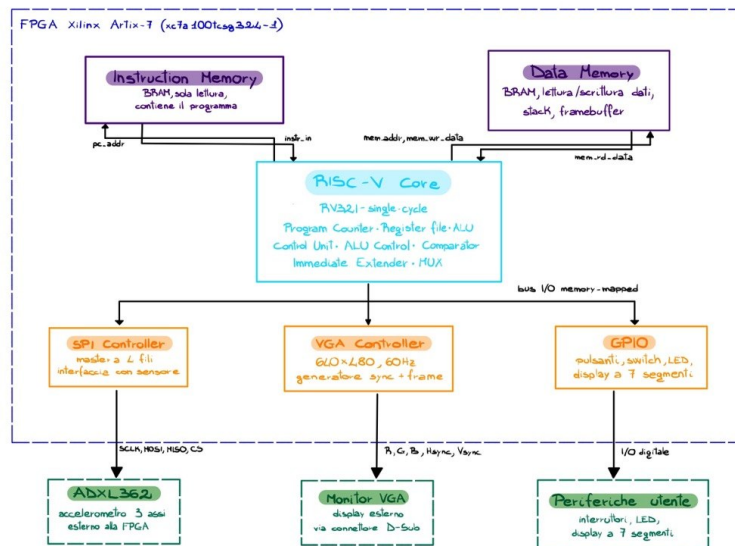


Figure 2.1: Block diagram

The RISC-V core is at the center of the architecture, connected to the memory blocks inside the FPGA. From a structural point of view, both the Data Memory and the Instruction Mem-

ory must provide data in a purely combinational way to respect the timing of the single cycle datapath. For this reason, they are mapped onto the Distributed RAM (LUT), keeping only the write operation synchronous.

To interact with the external environment, the processor uses a memory mapped communication bus. The core sends commands and exchanges data with the peripherals by executing standard read and write instructions in specific areas of the address space. An Address Decoder module analyzes the CPU requests and routes them to the appropriate controller.

Each hardware controller has a specific task:

- **SPI Controller:** manages the high-speed synchronous serial communication with the ADXL362 three-axis accelerometer, allowing the system to detect the board's tilt.
- **VGA Controller:** fetches the visual data calculated by the processor and generates the electrical and synchronization signals (HSYNC and VSYNC) needed to render the graphics on an external monitor.
- **GPIO (General Purpose Input/Output):** samples the input signals from buttons and switches, and drives the outputs to turn on the LEDs or show information on the seven-segment displays using time multiplexing.

2.2 Datapath diagram

The processor datapath follows a single cycle architecture based on the RISC-V ISA, featuring some custom optimizations compared to the standard model.

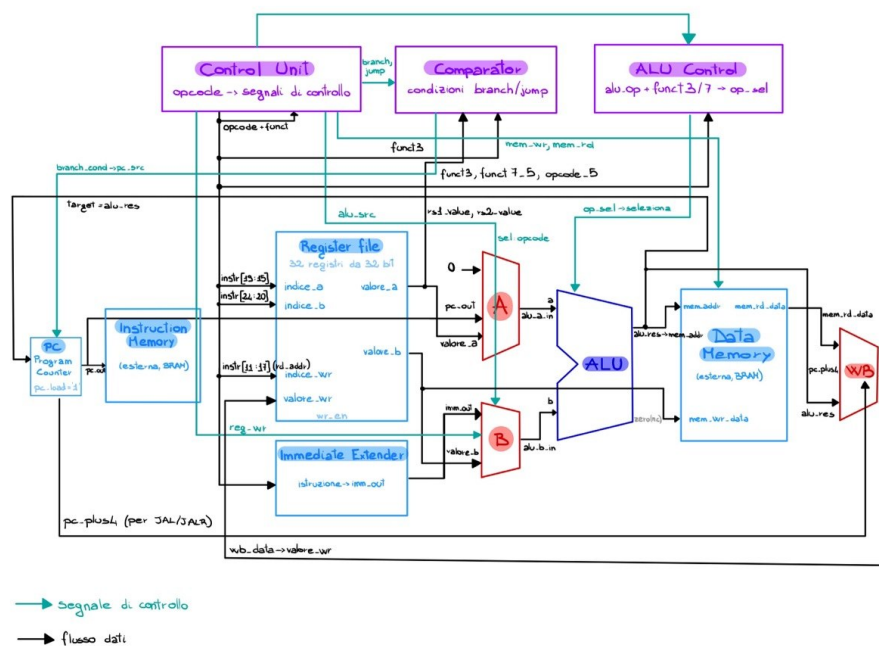


Figure 2.2: Datapath diagram

It illustrates the data paths and functional blocks that make up the CPU core. The processing of each instruction follows the five logical execution phases of a computer, condensed into a single clock cycle:

- **Instruction Fetch (IF):** The cycle starts with the Program Counter (PC), which provides the current address to the Instruction Memory to fetch the 32 bit code of the instruction to execute.

- **Instruction Decode (ID):** The Control Unit decodes the instruction Opcode to generate and route the control signals to the rest of the circuit. In parallel, the Register File provides the values read from the source registers, and the Immediate Extender extracts and sign extends any constant data (immediate) embedded in the instruction.
- **Execution (EX):** The data moves to the Arithmetic Logic Unit (ALU). Before entering, two selectors (multiplexers A and B) route the appropriate operands, dynamically choosing between the register values, the extended constant, or the PC address. At the same time, the Comparator module evaluates the logical conditions needed to determine the outcome of conditional branches.
- **Memory Access (MEM):** After the ALU processing, if the current instruction requires access to the main memory (Load/Store), the mathematical result of the operation is used as the address (Effective Address) to access the Data Memory.
- **Write-Back (WB):** The last phase is managed by the Write-Back multiplexer, which selects the data to write into the Register File. This can be the value read from the memory, the result computed by the ALU, or the return address saved during an unconditional jump. With this final synchronous write, the cycle ends, and the processor is ready to update the PC for the next instruction.

2.3 Instruction Format

In the RISC-V RV32I architecture, each instruction has a fixed length of 32 bits. The bit arrangement follows one of the six standard formats defined by the ISA specifications: R, I, S, B, U, and J. The instruction format determines how to read the individual fields and activates the correct hardware modules in the datapath.

Understanding these formats is essential for the logic synthesis of two main components: the Control Unit and the Immediate Extender. The Control Unit must interpret the Opcode to generate the multiplexing signals. At the same time, the Immediate Extender extracts the immediate values, which are scattered across non-contiguous positions, and correctly sign-extends them.

The table below summarizes the RV32I instructions implemented and supported by our custom CPU:

Type	Opcode	Instruction	Description
R-type	0110011	ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT, SLTU	Register-register operations
I-type	0010011	ADDI, ANDI, ORI, XORI, SLLI, SRLI, SRAI, SLTI, SLTIU	ALU with immediate
I-type	0000011	LW (e varianti)	Load from memory
I-type	1100111	JALR	Indirect jump
S-type	0100011	SW (e varianti)	Store to memory
B-type	1100011	BEQ, BNE, BLT, BGE, BLTU, BGEU	Conditional branches
U-type	0110111	LUI	Load upper immediate
U-type	0010111	AUIPC	Add upper immediate to PC
J-type	1101111	JAL	Jump and link

Table 2.1: RV32I instructions supported by the processor.

Chapter 3

Modules description

The processor memory system is composed of ten VHDL modules, each responsible for a specific function.

Module	Function
alu	32 bit combinational Arithmetic Logic Unit
register_file	Bank of 32 32-bit registers, with two asynchronous read ports and one synchronous write port.
control_unit	Primary opcode decoding and control signal generation logic network
alu_control	Logic network for decoding and selecting the ALU opcode
imm_extender	Module for extracting and sign extending immediate fields from various instruction formats.
program_counter	Synchronous register for tracking the current instruction address
comparator	Combinatorial module dedicated to the evaluation of jump conditions.
datapath	Top-level CPU module that instantiates and interconnects the datapath elements and control logic.
instr_memory	Instruction memory (ROM), implemented in distributed RAM (LUT) to ensure asynchronous reading.
data_memory	1024 word data memory (RAM), implemented as distributed memory (LUTRAM) with synchronous write and asynchronous read.

Table 3.1: System modules and memory.

3.1 Arithmetic Logic Unit (ALU)

The Arithmetic Logic Unit (ALU) is the computational core of the datapath. It is a combinational module that performs the arithmetic and logic operations of the processor. As shown in the VHDL entity, the interface has two 32 bit inputs for the operands (a and b), a 4-bit input to select the operation (s), and two outputs: the 32-bit result (alu_res) and a zero flag.

Since it is a combinational logic network, the module does not use synchronization signals and reacts immediately to input changes. The Comparator module will later use the zero flag to evaluate branch conditions.

```
1 entity alu is
2   Port (
3     a : in STD_LOGIC_VECTOR(31 downto 0);
4     b : in STD_LOGIC_VECTOR(31 downto 0);
5     s : in STD_LOGIC_VECTOR(3 downto 0);
6     alu_res : out STD_LOGIC_VECTOR(31 downto 0);
7     zero : out STD_LOGIC
8   );
9 end entity alu;
```

Listing 3.1: ALU Entity.

Several instructions share the same funct3 field and need the funct7 field to be uniquely identified (as in the ADD and SUB operations). To optimize the critical path and keep the

ALU as compact and fast as possible, we decided to separate this logic from the direct Opcode decoding. A dedicated external module (`alu_control`) handles the interpretation of the standard RISC-V fields. It provides the ALU with a custom, compact, and linear 4 bit selection code.

The ten operations implemented and supported by the ALU are:

0000	ADD (signed addition)
0001	SUB (signed subtraction)
0010	Bitwise AND
0011	Bitwise OR
0100	Bitwise XOR
0101	SLL (logical shift left)
0110	SRL (logical shift right)
0111	SRA (arithmetic shift right)
1000	SLT (set less than, signed)
1001	SLTU (set less than, unsigned)

Table 3.2: Encoding of ALU operations.

In VHDL, we used the IEEE standard library `numeric_std`. This library is essential for managing the difference between signed and unsigned logic types, which is a critical step for correctly synthesizing shift instructions.

To determine the shift amount for 32 bit data, we extract the 5 least significant bits of the second operand. For example, to synthesize the SRA (Shift Right Arithmetic) instruction in hardware, we applied the `shift_right` function to data cast as signed. This way, the library synthesis itself guarantees the correct sign extension. On the other hand, for the SRL (Shift Right Logical) instruction, casting the operand to unsigned forces the insertion of zero bits.

```

1 res <= STD_LOGIC_VECTOR(
2   shift_right(signed(a), to_integer(unsigned(b(4 downto 0))))
3 );

```

Listing 3.2: Example of SRA implementation.

3.2 Register File

The Register File is the internal working memory of the CPU. It implements a bank of 32 32-bit general purpose registers. At the architectural level, it is a multiport memory. It has two asynchronous read ports to extract the source operands at the same time, and one synchronous write port to save the results. As shown in the Entity, the interface includes the timing signals (`clk`, `rst`), three 5 bit address buses (which are necessary and sufficient to map $2^5 = 32$ registers), and three 32 bit data buses. There is also the `wr_en` enable signal. This signal ensures that writing to the destination register only happens on the rising edge of the clock.

```

1 entity register_file is
2   Port (
3     clk : in STD_LOGIC;
4     rst : in STD_LOGIC;
5     indice_a : in STD_LOGIC_VECTOR(4 downto 0);
6     valore_a : out STD_LOGIC_VECTOR(31 downto 0);
7     indice_b : in STD_LOGIC_VECTOR(4 downto 0);
8     valore_b : out STD_LOGIC_VECTOR(31 downto 0);
9     wr_en : in STD_LOGIC;
10    indice_wr : in STD_LOGIC_VECTOR(4 downto 0);
11    valore_wr : in STD_LOGIC_VECTOR(31 downto 0)
12  );

```

```
13 end entity register_file;
```

Listing 3.3: Register File Entity.

For the physical synthesis of these registers, we defined a custom VHDL type (`reg_array`) to represent an array of 32 32-bit elements. This choice makes the synthesis tools infer the structure as "distributed memory", mapping it onto the FPGA's Look Up Tables. Using BRAM hardware blocks would require a synchronous read (which introduces a one clock cycle delay). This delay is incompatible with the single cycle datapath, which needs zero architectural latency.

Finally, a specific feature of the RISC-V architecture requires the `x0` register to always hold the value zero. Therefore, we added specific checks to the module's logic: the synchronous process ignores any attempt to write to address zero, while during the combinational read phase, a multiplexer forces the output to the zero vector `0x00000000`.

```
1 valore_a <= (others => '0') when indice_a = "00000"
2     else regs(to_integer(unsigned(indice_a)));
```

Listing 3.4: Management of combinatorial reading and the `x0` register.

3.3 Control Unit

The Control Unit is the decision making block of the processor. It is a combinational network that decodes the Opcode of the incoming instruction to generate the control word. This word drives the multiplexer selection signals, the read and write enables for the memories, and the data flow in the datapath.

The tables below summarize the semantics of the output signals and the logical mapping of their values based on the specific instruction type.

Signal	Bit	Function
<code>reg_wr</code>	1	Enable writing to the register file.
<code>alu_src</code>	1	Select the second ALU operand (<code>rs2</code> or the extended immediate).
<code>mem_wr</code>	1	Enable writing to data memory.
<code>mem_rd</code>	1	Enable reading from data memory.
<code>mem_to_reg</code>	1	Select the write-back data (ALU result or memory output).
<code>branch</code>	1	Indicates a conditional branch instruction.
<code>jump</code>	1	Indicates an unconditional jump (JAL/JALR)
<code>alu_op</code>	2	Primary encoding of the ALU operation type

Table 3.3: Control signals generated by the Control Unit.

Type	<code>reg_wr</code>	<code>alu_src</code>	<code>mem_wr</code>	<code>mem_rd</code>	<code>mem_to_reg</code>	<code>branch</code>	<code>jump</code>	<code>alu_op</code>
R-type	1	0	0	0	0	0	0	10
I-type ALU	1	1	0	0	0	0	0	10
LOAD	1	1	0	1	1	0	0	00
STORE	0	1	1	0	0	0	0	00
BRANCH	0	0	0	0	0	1	0	00
JAL	1	1	0	0	0	0	1	00
JALR	1	1	0	0	0	0	1	00
LUI	1	1	0	0	0	0	0	11
AUIPC	1	1	0	0	0	0	0	00

Table 3.4: Control table by instruction type.

I use a two level decoding approach. The Control Unit only recognizes the operation class and generates the 2 bit primary code `alu_op`. It delegates the detailed decoding of the arithmetic or logic operation to the subordinate `alu_control` module, which evaluates the `funct3` and `funct7` fields. This separation of responsibilities makes the decoding hardware more compact and modular, and it reduces the critical path.

At the VHDL implementation level, we adopted two main practices:

1. We used symbolic constants to map the 7 bit RISC-V Opcodes, which improves readability.
2. We assigned default values (logic zero) to all output signals at the beginning of the sequential block. This technique is highly useful in the synthesis of combinational networks because it ensures that every signal is always driven, regardless of the executed case branch.

```

1 process(opcode, funct3, funct7_5)
2 begin
3     reg_wr <= '0';
4     alu_src <= '0';
5     mem_wr <= '0';
6
7     case opcode is
8         when OP_R_TYPE =>
9             reg_wr <= '1';
10            alu_op <= "10";
11
12     end case;
13 end process;
```

Listing 3.5: Structure of the combinatorial decoding process with latch prevention.

3.4 ALU Control

The ALU Control module completes the decoding system. It is a combinational logic network that translates the generic 2 bit `alu_op` signal along with the `funct3` and `funct7_5` fields extracted from the instruction. This process generates the 4-bit opcode (`op_sel`) that controls the ALU.

The division of responsibilities between the Control Unit and the ALU Control follows the two level decoding approach. The Control Unit determines the macro class of the operation (for example, a jump or a generic register to register operation). It then delegates the "fine decoding" to the ALU Control to properly drive the Function Unit.

This ensures high modularity. Decoupling the signals means that future ALU expansions will not affect the Control Unit's decision making logic, and vice versa.

During logic synthesis, it was necessary to carefully manage a critical issue related to the RISC-V ISA encoding formats. In R-type arithmetic instructions (like ADD and SUB), the operation distinction relies on bit 30 of the instruction. This bit corresponds to the `funct7_5` field.

The ADDI (I-type) instruction shares the same `funct3` ("000") as the ADD instruction. If the firmware executes an addition with a negative immediate (for example, -1), its two's complement representation puts a logic '1' at bit 30. Without proper checks, this overlap would make the hardware interpret the instruction as a subtraction, which would corrupt the execution.

To prevent this, the ALU Control interface was extended to sample the `opcode_5` signal as well. This signal is the sixth bit of the Opcode (it is '1' for R-type operations and '0' for I-type). As a result, the hardware infers a subtraction operation only under a strict logical condition:

```

1 when "000" =>
2     if funct7_5 = '1' and opcode_5 = '1' then
3         op_sel <= "0001"; -- SUB
```

```

4   else
5       op_sel <= "0000"; -- ADD
6   end if;

```

Listing 3.6: Safe handling of the subtraction operation in the ALU Control.

3.5 Immediate Extender

The Immediate Extender is a combinational module. It extracts and reconstructs the 32 bit immediate value from the input instruction word. It supports the five RISC-V ISA formats that use constants (I, S, B, U, and J). The module provides the correct sign extension by replicating the most significant bit of the immediate into the higher-order bits of the output vector. Regardless of the format, the sign bit is always located at bit 31 of the original instruction. This specific placement simplifies the critical path of the operation.

In the B and J formats, the immediate bits are arranged in non-contiguous positions within the instruction word. This encoding strategy minimizes hardware complexity. By keeping the immediate bits in the same relative positions across different formats, the design reduces both the number of multiplexers and the logic area required for decoding within the FPGA.

At the implementation level, the reconstruction of the immediates relies on the native concatenation operator of the VHDL language (&). This operator is combined with multiple assignment constructs to handle the sign bit extension.

Thanks to this choice, the concatenation operation does not infer any active logic gates (such as adders or shifters). Instead, it simply forces the synthesizer to perform a physical grouping and routing of the signals.

The decision block of the module is shown below.

```

1  process(istruzione, tipo_imm)
2  begin
3      case tipo_imm is
4          when "000" =>
5              imm_out <= (31 downto 12 => istruzione(31))
6                          & istruzione(31 downto 20);
7
8          when "001" =>
9              imm_out <= (31 downto 12 => istruzione(31))
10                        & istruzione(31 downto 25)
11                        & istruzione(11 downto 7);
12
13         when "010" =>
14             imm_out <= (31 downto 13 => istruzione(31))
15                       & istruzione(31)
16                       & istruzione(7)
17                       & istruzione(30 downto 25)
18                       & istruzione(11 downto 8)
19                       & '0';
20
21         when "011" =>
22             imm_out <= istruzione(31 downto 12)
23                       & (11 downto 0 => '0');
24
25         when "100" =>
26             imm_out <= (31 downto 21 => istruzione(31))
27                       & istruzione(31)
28                       & istruzione(19 downto 12)
29                       & istruzione(20)
30                       & istruzione(30 downto 21)
31                       & '0';
32

```

```

33     when others =>
34         imm_out <= (others => '0');
35
36     end case;
37 end process;

```

Listing 3.7: Combinatorial extraction and concatenation of immediates in RISC-V formats.

3.6 Program Counter

The Program Counter (PC) is the synchronous register responsible for tracking the memory address of the instruction currently being executed.

```

1 entity program_counter is
2     Port (
3         clk : in STD_LOGIC;
4         rst : in STD_LOGIC;
5         pc_load : in STD_LOGIC;
6         pc_src : in STD_LOGIC;
7         target : in STD_LOGIC_VECTOR(31 downto 0);
8         pc_out : out STD_LOGIC_VECTOR(31 downto 0);
9         pc_plus4 : out STD_LOGIC_VECTOR(31 downto 0)
10    );
11 end entity program_counter;

```

Listing 3.8: Program Counter Entity.

At each rising edge of the clock, in the absence of a reset signal, the internal state is updated by sampling the `pc_next` value. This address is calculated combinatorially through a multiplexer, according to the following decision logic:

$$pc_next = \begin{cases} \text{target} & \text{se } pc_src = '1' \text{ (branch preso o jump)} \\ pc_out + 4 & \text{altrimenti (avanzamento sequenziale)} \end{cases}$$

The standard increment of 4 units is necessary because the RV32I ISA instructions all have a fixed length of 32 bits and the architecture adopts byte-level memory addressing.

At the logic implementation level, the separation between the internal register and the output assignments is highlighted. Note how the `pc_plus4` output is explicitly calculated, which is essential to allow the return address to be saved in the Register File during subroutine jump instructions (JAL and JALR).

```

1 pc_next <= STD_LOGIC_VECTOR(unsigned(pc_reg) + 4) when pc_src = '0'
2     else target;
3 pc_out <= pc_reg;
4 pc_plus4 <= STD_LOGIC_VECTOR(unsigned(pc_reg) + 4);

```

Listing 3.9: Combinational logic for calculating jumps and PC+4.

3.7 Comparator

The Comparator is a combinational logic block that evaluates branch conditions. It directly fetches the `rs1` and `rs2` operands from the Register File. Depending on the decoding of the `funct3` field, it asserts the `branch_cond` signal. This signal then drives the `pc_src` input of the Program Counter to determine whether the branch is taken.

```

1 entity comparator is
2     Port (
3         rs1_value : in STD_LOGIC_VECTOR(31 downto 0);

```

```

4      rs2_value : in STD_LOGIC_VECTOR(31 downto 0);
5      branch   : in STD_LOGIC;
6      jump     : in STD_LOGIC;
7      funct3   : in STD_LOGIC_VECTOR(2 downto 0);
8      branch_cond : out STD_LOGIC
9  );
10 end entity comparator;

```

Listing 3.10: Comparator Entity.

The BLT, BGE, BLTU, and BGEU instructions require inequality evaluations. They operate on both signed and unsigned values. These arithmetic scenarios cannot be mathematically deduced from the **zero** flag alone.

For these reasons, the architecture includes a separate and dedicated hardware comparator. This choice relieves the ALU from performing a dummy subtraction at every branch. It also guarantees the correct combinational resolution of all six comparison cases defined by the standard.

As shown in the implementation snippet, the code uses the IEEE `numeric_std` standard library. It performs explicit casting to **signed** or **unsigned** types. This specific operation forces the synthesizer to infer the correct binary comparator topology.

```

1 when "100" =>
2     if signed(rs1_value) < signed(rs2_value) then premdi_branch <= '1'; end if;
3
4 when "110" =>
5     if unsigned(rs1_value) < unsigned(rs2_value) then premdi_branch <= '1'; end if;

```

Listing 3.11: Handling logical comparisons using explicit casting.

The output logic concludes with an inclusive OR operation between the branch result and the jump control signal, allowing unconditional jumps (JAL, JALR) and conditional jumps to be unified into the same data flow.

3.8 Datapath

This module represents the top hierarchical level of the RISC-V core. It instantiates and interconnects all the previously described operational modules (datapath elements and control logic) using a network of internal signals.

As shown in the Entity, the external memory subsystem is excluded from the internal architecture. Instead, it is placed at a higher hierarchical level. These memories are exposed through I/O ports. This clear separation of concerns improves the design's modularity. First, it allows testing the datapath alone using simulated memories. Furthermore, it makes it easier to instantiate the physical memory blocks during the synthesis phase, without requiring any changes to the processor's source code.

```

1 entity datapath is
2     Port (
3         clk : in STD_LOGIC;
4         rst : in STD_LOGIC;
5         instr_in : in STD_LOGIC_VECTOR(31 downto 0);
6         pc_addr : out STD_LOGIC_VECTOR(31 downto 0);
7         mem_addr : out STD_LOGIC_VECTOR(31 downto 0);
8         mem_wr_data : out STD_LOGIC_VECTOR(31 downto 0);
9         mem_rd_data : in STD_LOGIC_VECTOR(31 downto 0);
10        mem_wr_en : out STD_LOGIC;
11        mem_rd_en : out STD_LOGIC;
12        debug_wb_data : out STD_LOGIC_VECTOR(31 downto 0);
13        debug_rd_addr : out STD_LOGIC_VECTOR(4 downto 0);
14        debug_reg_wr : out STD_LOGIC

```

```

15 );
16 end entity datapath;

```

Listing 3.12: Datapath Entity.

The data flow routing within the architecture relies on three combinational multiplexers:

- **Multiplexer A (ALU Input):** Selects the first operand. It chooses among the data read from the source register, the current value of the Program Counter (needed to resolve JAL, branch, and AUIPC instructions), and the logical zero constant (used for the LUI instruction).
- **Multiplexer B (ALU Input):** Selects the second operand. It chooses between the data read from the source register and the 32 bit extended immediate. This choice strictly depends on the `alu_src` control signal.
- **Write-Back Multiplexer:** Determines the final data to save into the Register File. It routes the data fetched from the Data Memory, the return address `pc_plus4`, or the result computed by the ALU.

To minimize the logic resources on the FPGA, the design reuses the ALU to calculate the target address. By properly switching the input multiplexers, the ALU can calculate the sum $PC + imm$ (for branches, JAL, and AUIPC) or $rs1 + imm$ (for JALR). This approach effectively optimizes the logic area of the circuit.

To support the LUI and AUIPC instructions, the multiplexer for the ALU's A input has been extended to three inputs. LUI does not require reading any register, while AUIPC requires adding the Program Counter and the immediate. Including the Program Counter and the null constant at this stage guarantees the execution of these instructions. This setup avoids introducing custom operators or additional data paths.

```

1 alu_a_in <= pc_out when (opcode = OP_JAL or opcode = OP_BRANCH or opcode = OP_AUIPC) else
2   (others => '0') when opcode = OP_LUI else
3   valore_a;
4
5 alu_b_in <= imm_out when (alu_src = '1' or opcode = OP_BRANCH) else
6   valore_b;
7
8 wb_data <= mem_rd_data when mem_to_reg = '1' else
9   pc_plus4 when jump = '1' else
10  alu_res;

```

Listing 3.13: Implementation of concurrent assignments for routing multiplexers.

3.9 Instruction Memory

The instruction memory acts as a read only memory of 1024 32 bit words (4 KB). It hosts the processor's firmware, which is the RISC-V program already assembled and translated into machine language. At the VHDL code level, this program is described through the initialization of an array declared as a constant, a construct that tells the synthesizer that the data is immutable.

```

1 entity instr_memory is
2   Port (
3     clk : in STD_LOGIC;
4     addr : in STD_LOGIC_VECTOR(31 downto 0);
5     instr : out STD_LOGIC_VECTOR(31 downto 0)
6   );
7 end entity instr_memory;

```

Listing 3.14: Instruction memory Entity.

The read address `addr`, provided by the Program Counter, has a width of 32 bits. However, the ROM architecture only uses the [11:2] portion. Bits [1:0] are always zero because the RV32I instructions are word aligned (4 byte aligned). Extracting these 10 contiguous bits provides a correct and sufficient selection to address the $2^{10} = 1024$ physical cells available in the array.

The memory locations not actively used by the program are initialized to the default value 0x00000000. This precaution ensures that if the Program Counter accidentally diverges to an unprogrammed memory area, the processor encounters a predictable exception. This blocks the execution of anomalous behaviors.

The instruction extraction occurs combinationaly. Although the Entity exposes a `clk` signal for backward compatibility, the design does not use it to time the assignment.

Consequently, the Vivado synthesis tool will not infer BRAM hardware blocks, as they impose delayed synchronous reads. Instead, it maps the entire array of constants using the FPGA's Look Up Tables (LUTs). This implements the array as distributed memory (LUTRAM) with asynchronous reading. This design choice is essential to preserve the strict single cycle architecture. It ensures that the opcode becomes available in the exact same clock cycle that provides the address.

```
1 instr <= mem(to_integer(unsigned(addr(11 downto 2))));
```

Listing 3.15: Combinational reading and addressing of the memory array.

3.10 Data Memory

The Data Memory acts as a 1024 word, 32 bit (4 KB) random access memory, used by the processor for read and write operations. The memory array is declared using the signal construct, allowing its modification and dynamic updating during the execution of the program.

```
1 entity data_memory is
2   Port (
3     clk : in STD_LOGIC;
4     addr : in STD_LOGIC_VECTOR(31 downto 0);
5     wr_en : in STD_LOGIC;
6     rd_en : in STD_LOGIC;
7     wr_data : in STD_LOGIC_VECTOR(31 downto 0);
8     rd_data : out STD_LOGIC_VECTOR(31 downto 0)
9   );
10 end entity data_memory;
```

Listing 3.16: Data memory entity.

Compared to the ROM, the Entity has three additional ports to manage writing and conditional reading. The architecture uses two distinct and exclusive enable signals. The `wr_en` signal is activated during STORE instructions, and the `rd_en` signal is activated during LOAD instructions. When the processor executes instructions that do not access memory (for example, logic-arithmetic instructions), both signals remain at logic zero. This keeps the memory inactive, which reduces unnecessary switching and dynamic power consumption.

At the VHDL level, the write operation is implemented synchronously. It occurs within a process evaluated on the rising edge of the clock. Conversely, the read operation to extract data is described in a purely combinational (asynchronous) way. It sits outside the timed process and is evaluated only if the `rd_en` signal is active.

All memory cells are pre-initialized to 0x00000000 through the global assignment (`others => (others => '0')`). This assignment forces the Vivado tool to configure the hardware to boot in a cleared state when the board is powered on. This approach ensures deterministic system behavior at startup.

The standard use of the BRAM blocks integrated into the Artix-7 FPGA physically requires a synchronous read. This introduces a one-clock-cycle latency between the address assignment and the output data stability. To strictly preserve the single cycle architecture and avoid fatal timing misalignments in the datapath, the design intentionally makes the read operation asynchronous. Because of this modification, the Vivado synthesizer will not infer the BRAM hardware blocks. Instead, it will map the array using Look Up Tables to create a distributed memory (LUTRAM). This ensures that the data is accessible and usable in the exact same clock cycle it is requested.

Chapter 4

Processor Top-level

The `cpu.vhd` module represents the top-level entity of the processor and has the task of instantiating and interconnecting the three fundamental macro-blocks designed in the previous chapter: the Datapath, the Instruction Memory, and the Data Memory. Being a structural module, it does not infer any additional logic, but simply defines the routing of the internal signals necessary for the correct interfacing of the components.

4.1 Interface

```
1 entity cpu is
2   Port (
3     clk : in STD_LOGIC;
4     rst : in STD_LOGIC;
5     debug_pc : out STD_LOGIC_VECTOR(31 downto 0);
6     debug_instr : out STD_LOGIC_VECTOR(31 downto 0);
7     debug_wb_data : out STD_LOGIC_VECTOR(31 downto 0);
8     debug_rd_addr : out STD_LOGIC_VECTOR(4 downto 0);
9     debug_reg_wr : out STD_LOGIC
10  );
11 end entity cpu;
```

Listing 4.1: CPU Entity.

As can be seen from the Entity, the only functional ports necessary for the system's operation are the primary timing signals (`clk` and `rst`). The remaining ports are debug outputs, implemented to observe the evolution of the processor's internal state during simulations: the current value of the Program Counter, the instruction being executed, the data in the write back phase, the destination register, and the write enable signal. In a final version, these outputs can be removed or, alternatively, interfaced with integrated logic analyzers for in-system debugging.

4.2 Instantiated components

The table below lists the three macro-components instantiated within the top level module and their respective functional responsibilities.

Instance	Function
IMEM_inst	Instruction memory; receives the address from the Program Counter, returns the instruction.
DP_inst	Datapath; contains the PC, Register File, ALU, Control Unit, and all core modules.
DMEM_inst	Data memory; managed by LOAD/STORE instructions through the signals <code>mem_addr</code> , <code>mem_wr_data</code> , <code>mem_wr_en</code> , <code>mem_rd_en</code> .

Table 4.1: Top-level CPU components.

4.3 Benefits of the separation of responsibilities

The processor architecture was designed keeping the memory subsystem external and decoupled from the Datapath from the early design stages. This choice is reflected in the exposure of dedicated communication ports (`instr_in`, `mem_addr`, `mem_wr_data`, `mem_rd_data`, ecc.).

This approach consolidates the implementation of a Harvard architecture, characterized by separate buses for data and instructions. This separation pays off during the integration and testing phase: it ensures that the Datapath can be validated individually through testbenches using simulated memories, freeing it from the dependency on real BRAM hardware blocks.

The memories act as reusable, replaceable, or expandable modules without requiring any alteration to the processor core. Any future architectural developments will require changes limited to the CPU top-level module. This boundary between domains, known in software and hardware engineering as separation of concerns, represents a core principle for the development of modular, scalable, and robust designs.

Chapter 5

Functional verification

5.1 Core Modules Verification

Each processor module was subjected to tests aimed at verifying the logic functionalities described in the previous chapters. As expected, the testbenches are not limited to stimulating the inputs, but compare the observed outputs with the expected values using VHDL assertions. Any logical deviation between the produced result and the expected one generates an error message, interrupting the simulation and facilitating debugging.

The table below summarizes the tests performed for each macro-component of the system.

Module	What it verifies
alu	Arithmetic logic operations and zero flag assertion
register_file	Synchronous read/write and hardwiring of x0
control_unit	Control signals for each opcode
alu_control	Two-level fine decoding of the ALU opcode
imm_extender	Sign extension of the 5 immediate formats
program_counter	sequential advancement and PC jumps
comparator	Logical branch conditions (BEQ, BNE, BLT, ...)
instr_memory	Asynchronous reading of the ROM
data_memory	Synchronous write and asynchronous read with double enable

Table 5.1: Summary of the testbenches implemented for the validation of the core modules.

The most significant waveforms of each module are reported below.

Figure 5.1 shows the simulation of the ALU. It can be seen that the module executes the requested arithmetic-logic operations on the two input operands in a purely combinational way, returning the consistent result and setting the zero flag where the computation requires it.

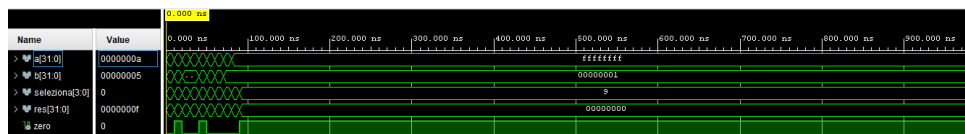


Figure 5.1: Timing simulation of the ALU

Figure 5.2 documents the access operations to the register file. Write transactions occur synchronously, while the correct implementation of the hardwiring can be observed: attempts to overwrite the x0 register are ignored, keeping its value fixed to logic zero..

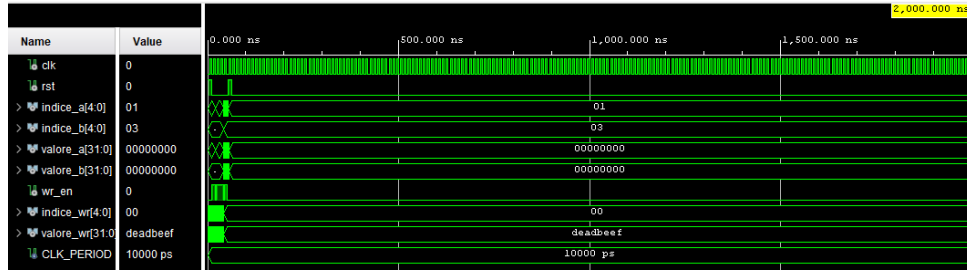


Figure 5.2: Register File Simulation.

In Figure 5.3, the combinational network of the Control Unit is validated. The waveforms demonstrate how the module reacts instantly to variations in the input `opcode`, driving the control signals necessary for the coordination of the datapath.

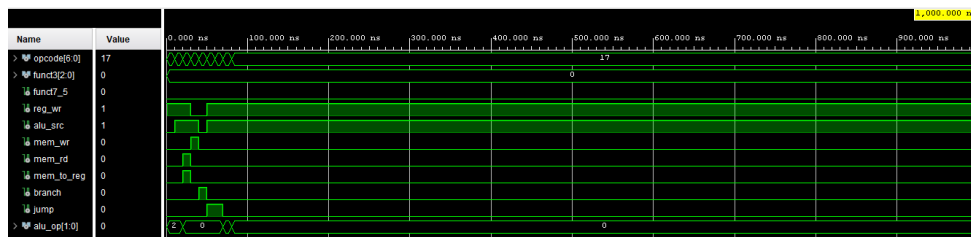


Figure 5.3: Control Unit simulation.

Figure 5.4 highlights the operation of the second decoding level. The module processes the `alu_op` signal in conjunction with the `funct3` and `funct7` fields, outputting the final selection code that will command the ALU's internal operations.

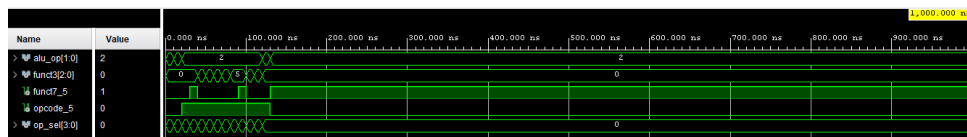


Figure 5.4: ALU Control simulation.

Figure 5.5 illustrates the simulation of the Immediate Extender. The waveforms confirm the correct extraction and sign extension of the 32-bit immediate value, operating according to the five decoding formats supported by the RISC-V architecture (I, S, B, U, J).

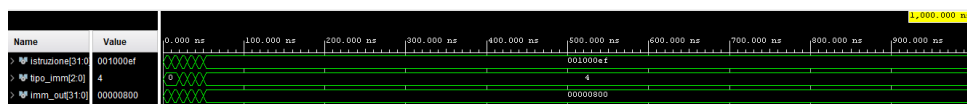


Figure 5.5: Immediate Extender simulation.

Figure 5.6 shows the sequential behavior of the Program Counter. The incremental advancement of 4 bytes per clock cycle during linear code execution can be observed, interspersed with abrupt non sequential updates corresponding to the calculation and activation of conditional and unconditional jumps.

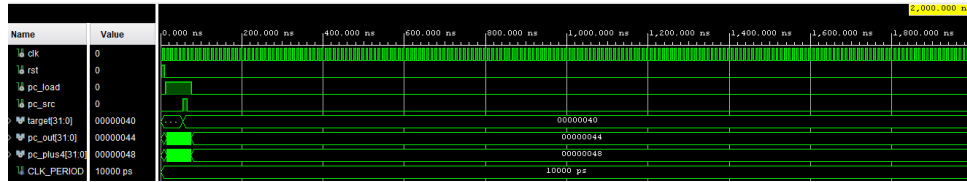


Figure 5.6: Program Counter simulation.

In Figure 5.7, the logic of the Comparator is analyzed. The transitions confirm how the module combinationally evaluates the six possible logical branch conditions (BEQ, BNE, BLT, etc.), asserting the jump signal depending on the input operands and the decoding of the funct3 field.

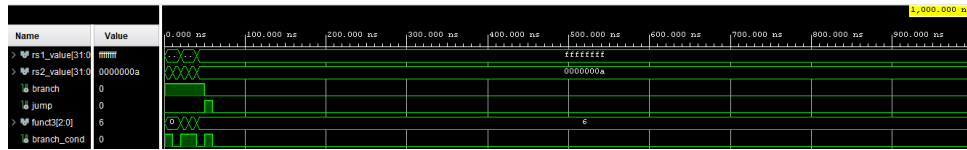


Figure 5.7: Comparator simulation.

Figure 5.8 documents the access to the Instruction Memory. Consistent with the distributed memory (LUTRAM) mapping described in the previous chapters, the read operation occurs asynchronously: the opcode is available on the output bus in the same clock cycle in which the address is presented, without any latency, as required by the single-cycle datapath.



Figure 5.8: Instruction memory simulation.

Finally, Figure 5.9 attests to the transactions on the Data Memory. The effectiveness of the double enable logic is highlighted: writes occur synchronously on the rising edge of the clock, while reads are purely combinational (asynchronous), consistent with the LUTRAM implementation adopted to preserve the single cycle architecture.

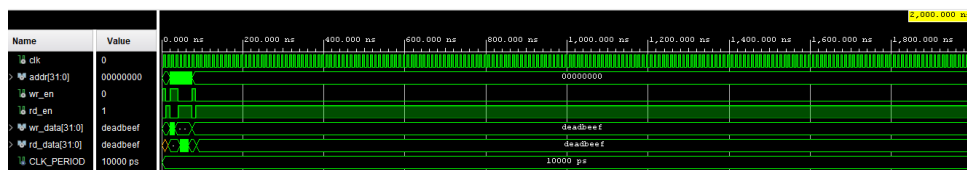


Figure 5.9: Data memory simulation.

5.2 Datapath verification

The validation of the Datapath was carried out by subjecting the module to the execution of a sequence of machine language instructions. The test micro program loads two known operands into two source registers, and then computes their sum, difference, and the logical AND and OR operations.

The data processing flow was traced and validated, verifying the correct propagation of the signals through the stages of the single-cycle architecture: instruction fetch, decode, execute, and result consolidation.

```
Note: === Reset rilasciato, inizio esecuzione ===
Note: Dopo ciclo 1: PC=4 ultima scrittura: rd=2 val=10
Note: Dopo ciclo 2: PC=8 ultima scrittura: rd=3 val=15
Note: Dopo ciclo 3: PC=12 ultima scrittura: rd=4 val=5
Note: Dopo ciclo 4: PC=16 ultima scrittura: rd=5 val=0
Note: Dopo ciclo 5: PC=20 ultima scrittura: rd=6 val=15
Note: Dopo ciclo 6: PC=24 ultima scrittura: rd=0 val=0
Note: === TEST COMPLETATO ===
```

Figure 5.10: Results of the complete datapath simulation.

Figure 5.10 shows the results generated at the end of the simulation. The correspondence between the progression of the operating cycles and the write transactions in the Register File can be observed, confirming that the values computed by the ALU are correctly routed and saved in the destination registers at each clock cycle.

5.3 CPU verification

The final level of validation is the top level instance of the CPU, which integrates the Datapath with the memory subsystem. Upon the release of the asynchronous reset signal, the CPU starts the execution of the firmware pre-loaded in the Instruction Memory, fetching an opcode for each cycle and updating the state of the Register File.

Table 5.2 documents the correspondence between the sequence of executed instructions and the values observed on the debug ports.

Cycle	Instruction	Logical Calculus	Written Value
1	addi x1, x0, 5	$0 + 5 = 5$	x1 = 5
2	addi x2, x0, 10	$0 + 10 = 10$	x2 = 10
3	add x3, x1, x2	$5 + 10 = 15$	x3 = 15
4	sub x4, x2, x1	$10 - 5 = 5$	x4 = 5
5	and x5, x1, x2	$5 \wedge 10 = 0$	x5 = 0
6	or x6, x1, x2	$5 \vee 10 = 15$	x6 = 15

Table 5.2: Results of the test program execution on the complete CPU.

As confirmed by the analysis of the figure below, all the values sampled on the output lines match the mathematical expectations.

```
Note: === Reset rilasciato, inizio esecuzione ===
Note: Ciclo 1: PC=0x00000004 istr=0x00500093 scrive x1 val=5
Note: Ciclo 2: PC=0x00000008 istr=0x00A00113 scrive x2 val=10
Note: Ciclo 3: PC=0x0000000C istr=0x002081B3 scrive x3 val=15
Note: Ciclo 4: PC=0x00000010 istr=0x40110233 scrive x4 val=5
Note: Ciclo 5: PC=0x00000014 istr=0x0020F2B3 scrive x5 val=0
Note: Ciclo 6: PC=0x00000018 istr=0x0020E333 scrive x6 val=15
Note: Ciclo 7: PC=0x0000001C istr=0x00000013 scrive x0 val=0
Note: Ciclo 8: PC=0x00000020 istr=0x00000013 scrive x0 val=0
Note: Ciclo 9: PC=0x00000024 istr=0x00000013 scrive x0 val=0
Note: Ciclo 10: PC=0x00000028 istr=0x00000013 scrive x0 val=0
Note: === TEST COMPLETATO ===
```

Figure 5.11: Results of the full-CPU timing simulation.

Chapter 6

Hardware peripherals

The system features three dedicated hardware peripherals. They are external to the processor core but synthesized within the FPGA logic. The architecture integrates an SPI controller to interface with the ADXL362 3 axis accelerometer. It also includes a VGA controller for graphics rendering on a monitor, and a GPIO block to manage switches, buttons, LED indicators, and the seven-segment display.

An I/O bus using the memory mapped technique manages the communication between the peripherals and the CPU. Therefore, the processor interprets the external devices as standard memory locations. The CPU performs standard read or write transactions on specific, predefined address ranges. These transactions physically translate into sending configuration commands or acquiring data from the sensors.

6.1 Controller SPI

The SPI (Serial Peripheral Interface) controller represents the hardware interface delegated to communication with the ADXL362 3 axis accelerometer, integrated on the Nexys 4 DDR board. This sensor detects the tilt of the platform in real time along the three Cartesian axes, providing the processor with the fundamental kinematic data for the interactive control of the ball inside the maze.

SPI is a synchronous serial communication protocol operating in a master-slave topology, based on four distinct signal lines: SCLK (serial clock), MOSI (for data transmission to the peripheral), MISO (for data reception from the sensor), and CS (active low, necessary to select and enable the slave). Communication occurs in full-duplex mode: at each active edge of the clock, the master transmits one bit on the MOSI line and samples one in reception on the MISO line.

The controller was configured to operate in "Mode 0", characterized by the phase and polarity parameters CPOL=0 and CPHA=0. In this configuration, the clock line remains at a logic low level during the idle phases; bits are sampled on the rising edge of SCLK and updated on the falling edge. The first useful bit is presented by the master immediately after the assertion of the Chip Select signal.

The sequential control logic of the module is implemented through a Finite State Machine structured in four essential operating states: IDLE (idle state and waiting for the start command), ASSERT_CS (assertion of the Chip Select and generation of a guard time delay equal to half a period), SHIFT (synchronized transfer of the 8-bit payload), and DONE (release of the Chip Select and assertion of the transaction completion flag).

The frequency of the serial clock SCLK is parametrically scalable through the Generic construct CLK_DIV, which acts as a frequency divider of the system clock. By setting CLK_DIV = 50 starting from a primary clock of 100 MHz, an SCLK of 1 MHz is obtained, an optimal and conservative value compared to the 8 MHz limit frequency supported by the accelerometer.

```
1 entity spi_master is
2   Generic (
3     CLK_DIV : positive := 50
4   );
5   Port (
6     clk : in STD_LOGIC;
7     rst : in STD_LOGIC;
```



```

8      start : in STD_LOGIC;
9      tx_data : in STD_LOGIC_VECTOR(7 downto 0);
10     rx_data : out STD_LOGIC_VECTOR(7 downto 0);
11     busy : out STD_LOGIC;
12     done : out STD_LOGIC;
13     spi_sclk : out STD_LOGIC;
14     spi_mosi : out STD_LOGIC;
15     spi_miso : in STD_LOGIC;
16     spi_cs_n : out STD_LOGIC
17 );
18 end entity spi_master;

```

Listing 6.1: Controller SPI Master entity.

For the validation of the isolated module, a testbench was developed that instantiates a behavioral model of a dummy SPI slave. It is programmed to respond to the reception of the payload sent by the master by transmitting a known test byte. The simulations confirm the validity of the protocol: the byte sampled by the master at the end of the transaction corresponds to the sequence emitted by the slave, verifying the effectiveness of the full duplex transmit-receive mechanism.

Figure 6.1 captures the logic analyzer, highlighting the timing of the serial signals during a complete byte transfer.

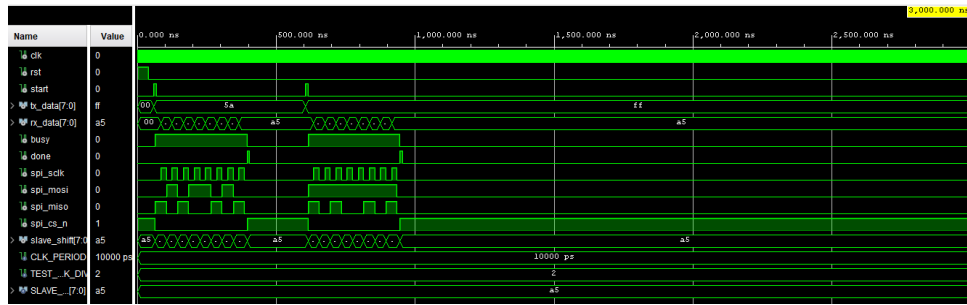


Figure 6.1: Timing simulation of a complete full-duplex SPI transfer.

6.2 Controller VGA

The VGA controller is used to generate the signals necessary to drive an external monitor through the board's D-Sub connector, ensuring a standard resolution of 640x480 pixels with a refresh rate of 60 Hz.

The image scanning on a VGA monitor occurs pixel by pixel, from left to right and from top to bottom. This is controlled by two synchronization signals: HSYNC (horizontal, signals the end of a line) and VSYNC (vertical, indicates the end of an entire frame). The protocol requires the insertion of blanking intervals because, historically, they were used to allow the repositioning of the electron beam in cathode ray tube monitors. These are divided into front porch, sync pulse, and back porch.

Specifically for the 640x480 resolution at 60 Hz, a line consists of 800 pixel clock cycles. Similarly, a complete frame is composed of 525 lines. The pixel clock frequency necessary to meet these specifications is $800 \times 525 \times 60 \approx 25$ MHz, obtained by dividing the 100 MHz main system clock by four.

The VGA subsystem was designed with a modular architecture. The `vga_sync` module integrates the horizontal and vertical counters, taking care of generating the HSYNC and VSYNC signals. It calculates the physical coordinates of the current pixel during scanning and produces the `vide_on` signal, which discriminates whether the virtual "beam" is within the active display area or in the blanking zone.

A second logic module receives the coordinates in real-time and generates the RGB values to be sent to the monitor. During the hardware testing phase, a pattern generator capable of drawing eight colored vertical bands was implemented; in the final version, it is replaced by `vga_ball`, dedicated to the dynamic rendering of the maze background and the ball.

The hardware of the Nexys 4 DDR board assigns 4 bits to each color channel (red, green, blue), offering a 12-bit color depth (4096 possible colors). The color channels are asynchronously forced to zero (black color) whenever the `video_on` signal is deactivated (during the blanking periods).

```

1 entity vga_sync is
2   Port (
3     clk : in STD_LOGIC;
4     rst : in STD_LOGIC;
5     hsync : out STD_LOGIC;
6     vsync : out STD_LOGIC;
7     pixel_x : out STD_LOGIC_VECTOR(9 downto 0);
8     pixel_y : out STD_LOGIC_VECTOR(9 downto 0);
9     video_on : out STD_LOGIC
10  );
11 end entity vga_sync;
```

Listing 6.2: VGA sync generator entity.

The analysis confirms the correct timing of the synchronization signals. During the simulation of a complete frame, it is possible to observe 525 HSYNC pulses (one for each scanned line) and a single VSYNC pulse. Figure 6.2 shows the waveforms of the `vga_sync` module testbench, highlighting the correct delimitation of the lines and the frame.

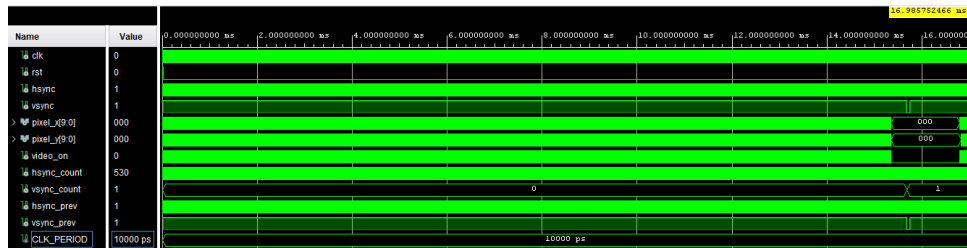


Figure 6.2: Waveforms of the VGA synchronization signals generated by the `vga_sync` module.

The interaction with the color pattern generator was tested by verifying the RGB transitions corresponding to the passage of the pixel clock through the eight predefined bands. Figure 6.3 attests to the reliability of the color multiplexing logic.

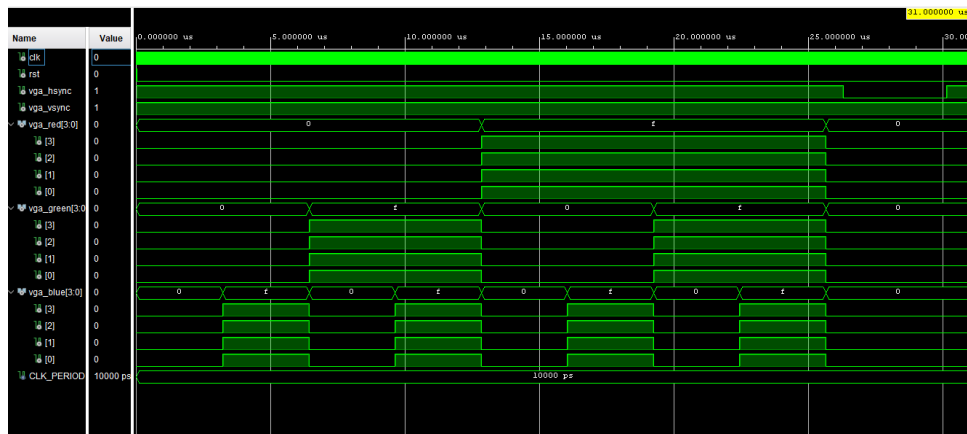


Figure 6.3: Generation of RGB channels for the eight color bands.

6.3 GPIO & Seven-segment display

The GPIO (General Purpose Input/Output) module represents the hardware interface dedicated to managing the basic peripherals integrated on the Nexys 4 board. It controls the 16 slide switches, the 5 directional buttons, the 16 LED indicators, and the eight digit seven segment display.

The module does not require dedicated instructions in the processor's ISA, but exposes four 32 bit registers to the CPU, accessible via LOAD and STORE instructions. Table 6.1 illustrates the map of the offset addresses and the related access directives.

Offset	Register Description	Access
0x0	Combinatorial state of the 16 switches	Read-only
0x4	Combinatorial state of the 5 push-buttons	Read-only
0x8	16-LED power-on value	Write-only
0xC	Hexadecimal encoding for the display	Write-only

Table 6.1: Register map of the GPIO subsystem.

For the hardware implementation of the seven-segment display, in order to minimize the number of pins used, the eight digits physically share the same seven line data bus. To display independent values in each position, the module adopts a time multiplexing technique: the hardware sequentially activates a single digit at a time, driving its respective segment configuration in a circular manner. By maintaining a sufficiently high refresh rate, the phenomenon of persistence of vision causes the user to perceive all eight digits as being illuminated simultaneously.

At the electrical level, both the segments and the selection anodes operate in active-low logic: a logic '0' level turns on the associated component. A dedicated decoding module (`hex_to_7seg`) handles the translation of each nibble (4-bit hexadecimal digit) into the correct topological combination of the seven segments.

```

1 entity gpio is
2   Port (
3     clk : in STD_LOGIC;
4     rst : in STD_LOGIC;
5     addr : in STD_LOGIC_VECTOR(31 downto 0);
6     wr_data : in STD_LOGIC_VECTOR(31 downto 0);
7     rd_data : out STD_LOGIC_VECTOR(31 downto 0);
8     wr_en : in STD_LOGIC;
9     switches : in STD_LOGIC_VECTOR(15 downto 0);
10    buttons : in STD_LOGIC_VECTOR(4 downto 0);
11    leds : out STD_LOGIC_VECTOR(15 downto 0);
12    seg : out STD_LOGIC_VECTOR(6 downto 0);
13    an : out STD_LOGIC_VECTOR(7 downto 0);
14    dp : out STD_LOGIC
15  );
16 end entity gpio;
```

Listing 6.3: GPIO peripheral controller entity.

The functional validation of the module was carried out using a testbench. In a first phase, the simulation confirmed the perfect execution of the multiplexing cycle: by sending the test word 0x1234ABCD to the display register, the sequential activation of the eight digits on their respective anodes can be observed. Figure 6.4 documents the temporal coherence of this circular scanning.

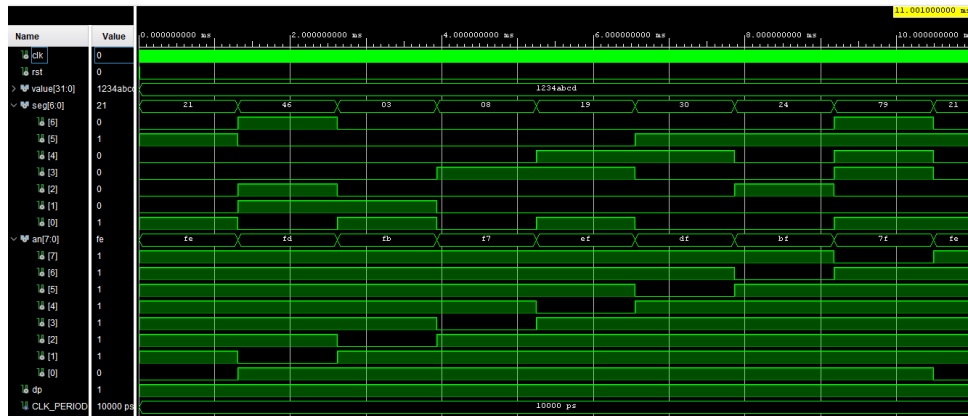


Figure 6.4: Timing simulation: seven-segment display anode multiplexing cycle.

Finally, Figure 6.5 shows an overview of the simulation of the complete GPIO subsystem, highlighting the correct routing of input data (reactive reading of switches and buttons) and output data (asynchronous update of the LEDs and the display register) in response to the stimuli from the CPU data bus.

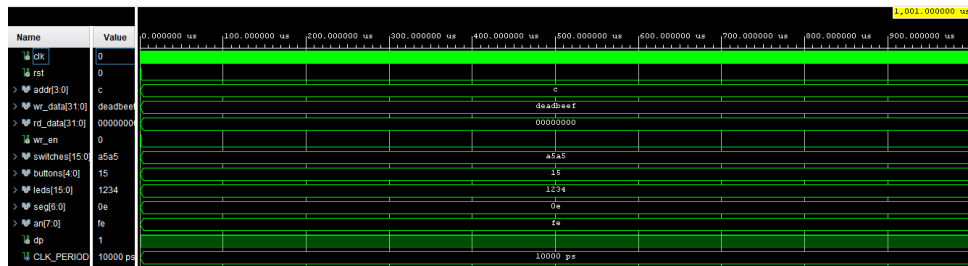


Figure 6.5: GPIO module simulation: reading switches and push-buttons, writing to LEDs and displays.

Chapter 7

System integration

After verifying the processor and the peripherals, the final step is to integrate them into a single system. The `system_top` module represents the top level of the entire project. It connects the CPU, the data memory, and the three peripherals (GPIO, SPI, and VGA) using a memory mapped I/O bus.

7.1 Memory-mapped I/O

The RISC-V processor communicates with the outside world using architectural LOAD and STORE instructions. These instructions allow the system to read and write data by specifying an address. The memory mapped I/O technique uses this feature. It assigns a specific range of physical addresses to each hardware peripheral. Performing a read or write transaction on these locations means communicating directly with the corresponding peripheral, rather than with the main memory.

In this setup, the processor does not need special instructions for Input/Output management. From the CPU's perspective, the state of the LEDs, the display digits, the SPI controller commands, and the VGA controller graphics registers all appear as regular RAM memory locations.

7.2 Address Decoder & Map Memory

The address decoder is a combinational network that examines the high-order bits of the address generated by the CPU and routes the request to the correct resource. The decoder performs two tasks:

- during write operations, it enables only the peripheral whose address range matches (via filtered enable signals);
- during read operations, a multiplexer selects the data coming from the addressed resource and returns it to the CPU.

Table 7.1 summarizes the assignment of addresses to the different system resources. The distinction is made on bits 29 and 28 of tSystem memory map.

Interval	Resource	bit 29:28
0x00000000+	RAM dati	00
0x10000000+	GPIO	01
0x20000000+	SPI	10
0x30000000+	VGA	11

Table 7.1: System memory map.

Within each range, the low order bits of the address select the specific peripheral register. For example, in the GPIO area, the offset `0x8` corresponds to the LED register. In the VGA area, the offsets `0x0` and `0x4` set the X and Y coordinates of the ball, respectively.

7.3 Integrated system audit

To validate the operation of the complete system, a machine language program was developed and preloaded into the instruction memory, which performs the following operations:

- writing the hexadecimal value 0x00FF to the LED register via the GPIO module;
- updating the value shown on the seven-segment display;
- initiating an SPI serial transfer by sending a test byte to the accelerometer;
- setting the physical coordinates of the ball within the VGA subsystem.

The simulation results confirm that the entire peripheral ecosystem responds accurately to the CPU's directives. The figure below shows the textual output generated by the testbench. This output confirms that all checks passed successfully.

```

=== Reset rilasciato, la CPU inizia a eseguire ===
LED = 0x255 (=0x00FF)
LED OK (0x00FF)
Posizione pallina impostata: X=100, Y=150
In attesa del completamento SPI...
=== Osservazione completata ===
ball_x_reg=100 / ball_y_reg=150
vga_hsync e vga_vsync generano i sincronismi

```

Figure 7.1: Simulation console

The analysis of the timing diagram certifies the correct interfacing at the physical signal level. From the waveforms, it is possible to observe the simultaneous coexistence of the operations: the static update of the GPIO output, the SPI serial transaction (through the exact timing of `spi_sclk`, `spi_mosi`, and `spi_cs_n`), and the uninterrupted generation of the VGA video synchronizations (`vga_hsync` and `vga_vsync`).

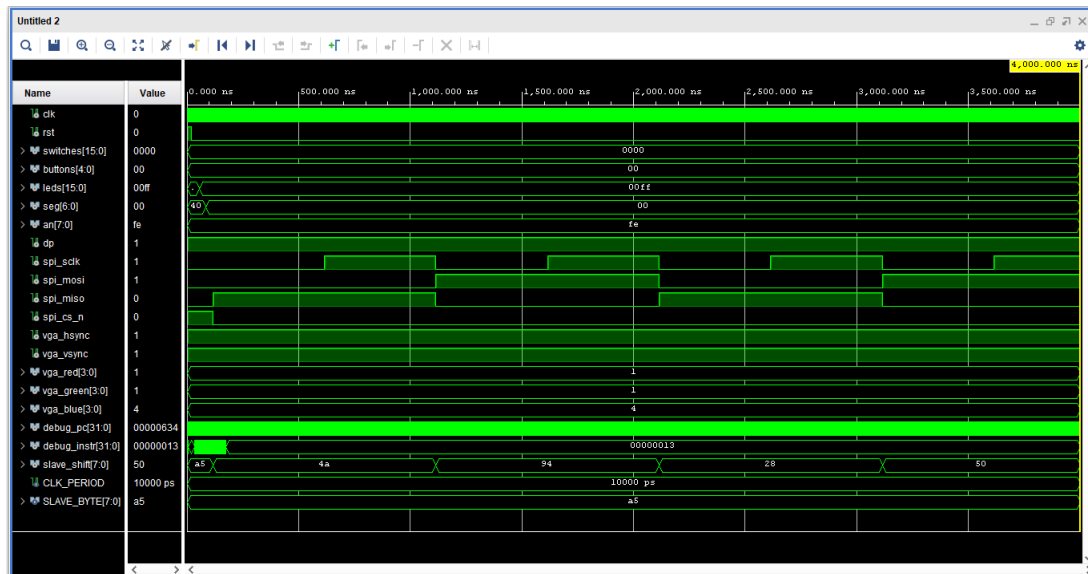


Figure 7.2: Temporal analysis of the System-on-Chip.

This certifies the full success of the architectural integration: a single cycle RISC-V processor driving an ecosystem of hardware peripherals using a unified memory mapped bus.

Chapter 8

Game software

The final test of the RISC-V architecture required the development of a bare-metal firmware. Using the memory-mapped I/O topology, an interactive video game (maze) was designed. It gets the tilt data from the accelerometer via SPI, calculates the kinematics of the ball, and dynamically updates its coordinates on the VGA monitor.

8.1 Firmware Architecture and Game Logic

The software is written in C. It interfaces directly with the physical hardware registers. To implement the memory mapped I/O, the code defines volatile pointers. These pointers map to the exact physical addresses established in the Address Decoder. Using the volatile directive is crucial. It prevents the compiler from optimizing and removing repeated reads or writes. This guarantees that every C memory access correctly translates into a transaction on the data bus.

The system interacts with the ADXL362 accelerometer through low-level functions. These functions hide the complexity of the serial protocol from the video game. The `spi_transfer()` send function implements a polling (active waiting) mechanism on the status bit of the `SPI_STATUS` register. The CPU waits in a while loop until the hardware peripheral lowers the busy flag. This signals the completion of the full duplex transfer.

Calculating the game physics, specifically simulating inertia and friction, was the most complex algorithmic choice. The core uses the base RV32I instruction set, which lacks hardware multipliers and dividers. Because of this, the mathematical logic relies solely on addition, subtraction, and bit shift operations. Using right and left bit-shifts creates a solid fixed point arithmetic implementation. This approach is essential to scale the velocity vector and simulate dynamic friction without overloading the ALU.

Also, the software introduces a deadzone on the raw acceleration data by ignoring readings between -4 and +4. This filters the natural noise of the MEMS sensor and prevents the ball from drifting unintentionally.

Finally, the collision logic for the maze walls and holes relies on the combined evaluation of rectangular intersections.

```
1 #define LEDS (*(volatile unsigned int *)0x10000008)
2 #define SPI_TX (*(volatile unsigned int *)0x20000000)
3 #define SPI_STATUS (*(volatile unsigned int *)0x20000004)
4 #define SPI_RX (*(volatile unsigned int *)0x20000008)
5 #define SPI_CS (*(volatile unsigned int *)0x2000000C)
6 #define VGA_BALL_X (*(volatile unsigned int *)0x30000000)
7 #define VGA_BALL_Y (*(volatile unsigned int *)0x30000004)
8
9 unsigned int spi_transfer(unsigned int data) {
10     SPI_TX = data;
11     while ((SPI_STATUS & 0x1) != 0) {}
12     return SPI_RX;
13 }
14
15 int main() {
16     accel_init();
17     int pos_x = 50 << 5;
18     int pos_y = 50 << 5;
19     int vel_x = 0, vel_y = 0;
20 }
```

```

21  while (1) {
22      int raw_y = read_accel(0x08);
23      int raw_x = read_accel(0x09);
24
25      if (raw_x > -4 && raw_x < 4) raw_x = 0;
26      if (raw_y > -4 && raw_y < 4) raw_y = 0;
27
28      int acc_x = -(raw_x >> 3);
29      int acc_y = raw_y >> 3;
30
31      vel_x = vel_x + acc_x - (vel_x >> 3);
32      vel_y = vel_y + acc_y - (vel_y >> 3);
33
34      int next_px = (pos_x + vel_x) >> 5;
35      int next_py = (pos_y + vel_y) >> 5;
36
37      if (wall_hit(next_px, next_py, 0, 80, 160, 180) || /* ... */) {
38          vel_x = 0;
39          vel_y = 0;
40          next_px = pos_x >> 5;
41          next_py = pos_y >> 5;
42      }
43
44      pos_x = next_px << 5;
45      pos_y = next_py << 5;
46
47      VGA BALL_X = next_px;
48      VGA BALL_Y = next_py;
49      delay();
50  }
51  return 0;
52 }

```

Listing 8.1: Firmware C extract.

8.2 Compilation, Boot, and Hardware Initialization Flow

After verifying the processor and the peripherals, the final step was to translate the firmware into executable hardware instructions. A GCC based cross compilation flow was adopted for this task.

During the first compilation attempts, the designed CPU could not execute the resulting binary. An analysis of the hexadecimal dump revealed the issue. The default compiler was emitting 16 bit compressed instructions (C extension) and 64 bit RV64 code. The RV32I datapath cannot decode these formats.

To solve this problem, it was necessary to strictly define the target architecture and the ABI (Application Binary Interface) during the compilation phase. This setup also included adding the space optimization flag (-Os):

```

1 riscv64-unknown-elf-gcc -march=rv32i -mabi=ilp32 -Os

```

Listing 8.2: GCC compilation options for the pure RV32I architecture.

The system operates in a bare metal environment without an Operating System. Therefore, the C compiler does not automatically provide hardware startup routines. It only translates the individual functions. Before calling the main function, the system must initialize the Stack Pointer. Otherwise, it would write local variables and return addresses to random memory locations. This would cause a system crash.

For this reason, a custom boot micro-sequence was written manually in pure Assembly language. This sequence is located at address 0x0 of the Instruction Memory:


```

1 _start:
2     li sp, 4092 # Stack pointer in cima alla RAM dati (allineato)
3     jal ra, main # Salto all'entry point del programma C
4 _halt:
5     jal ra, _halt # Loop infinito di sicurezza (se il main ritorna)

```

Listing 8.3: Assembly boot sequence with stack initialization and safety loop.

As seen in the code, the Stack Pointer (sp) is set to the value 4092. This is the last aligned word available in the 4 KB Data Memory. This setup allows the stack to grow correctly downwards.

An online RISC-V assembler was used to translate the entire code block (Boot sequence plus compiled Firmware) into its corresponding Hex Dump (machine language). As required for the FPGA implementation, the firmware is not loaded from an external source. Instead, it is embedded directly within the System on Chip.

The hexadecimal codes generated by the assembler were used to initialize the VHDL array. This array describes the instr_memory, which is implemented as distributed memory (LUTRAM) with asynchronous reading. When the board powers on, or when the hardware reset signal is released, the CPU fetches the instruction at location zero. It then starts executing the game.

8.3 System Simulation and Pre-Implementation Validation

It is essential to include timing simulations to confirm the architecture's stability and overall performance. To validate the correct operation of the entire SoC, an extended behavioral simulation (lasting 1 ms) of the integrated system was performed.

This computer based validation phase is a crucial step. It verified the exact execution of the firmware. It also successfully tested the interaction between the CPU and the peripheral modules within a controlled environment.

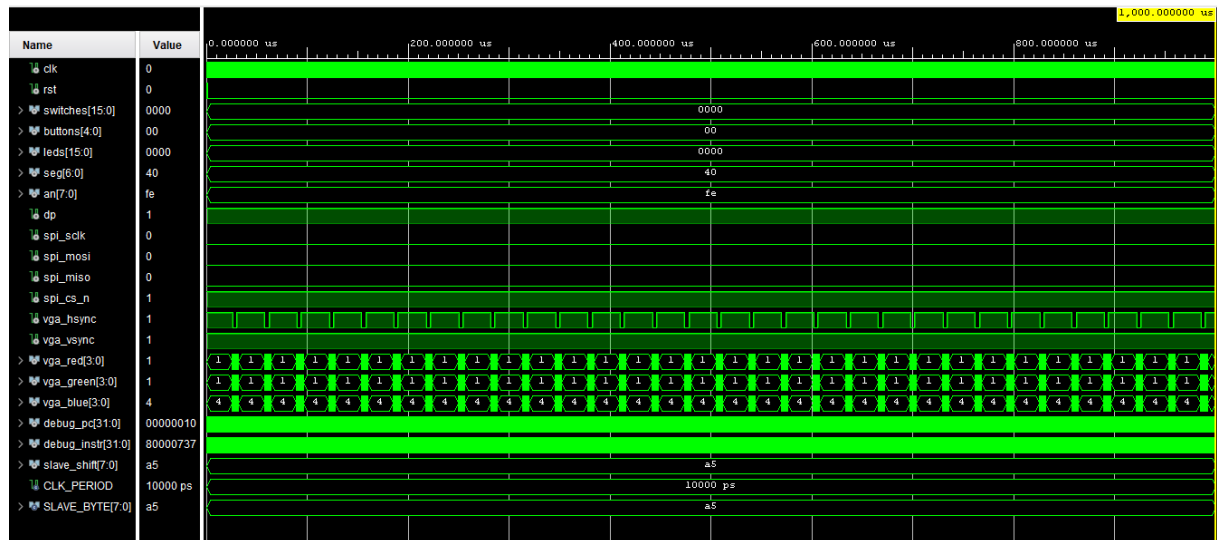


Figure 8.1: System simulation waveforms (1 ms).

The timing diagram analysis confirms the successful hardware/software integration:

- **Firmware Execution:** The debug_pc bus changes constantly. This shows that the Program Counter has entered the infinite game loop. The CPU processes the physical game logic without encountering stalls or freezes.

- **Automatic SPI Communication:** The `spi_cs_n` signal shows periodic low-level assertions. This proves that the CPU autonomously and continuously queries the external accelerometer using C routines. These routines correctly manage the memory mapped registers and wait for the serial bus transaction to complete.
- **Independent VGA Rendering:** The `vga_hsync` signal pulses are continuously generated in the background. Simultaneously, the ALU modifies the ball's coordinates. These coordinates update in response to the simulated acceleration data provided by the SPI slave in the testbench.

These results demonstrate that the SoC is stable. It is now ready to be synthesized and programmed onto the physical board.

Chapter 9

FPGA implementation

After completing the simulation verification, the project was transferred to the physical Nexys 4 board. This phase required writing the constraint file and adapting the top level module. It also involved resolving three specific issues that emerged with the real hardware. These issues included the timing misalignment of the synchronous memories, the failure to meet timing constraints (Setup Violations), and the correct propagation of the clock domains.

9.1 Physical Constraints and Top-Level Adaptations

The `nexys4_labirinto.xdc` configuration file maps each logic port of the top-level entity to the physical pins of the Artix-7 FPGA. The constrained signals include the 100 MHz system clock (pin E3) and the VGA controller signals (synchronization and 4-bit RGB channels). They also include the GPIO block (LEDs, seven-segment display, switches, and buttons) and the four wires of the SPI interface for the onboard ADXL362 accelerometer.

The physical CPU reset button on the Nexys 4 board provides a low logic level ('0') when pressed. However, the processor's internal architecture is designed to react to an active-high reset. An inverter added to the top-level module (`rst_int <= not rst`) resolves this issue. This solution preserves the internal sequential logic without any changes.

An initial implementation attempt failed prematurely with an "IO placement is infeasible" synthesis error. The cause was the `debug_pc` and `debug_instr` buses, which total 64 bits. They required the allocation of 64 physical I/O pins. This number is much higher than the available free pins on the FPGA. To allow the routing phase to complete, these signals were removed from the top-level interface used for the final synthesis.

9.2 Resolution of Architectural Issues

During testing, the architecture showed anomalous behavior. The Program Counter assumed unaligned values. Furthermore, the `jal` instruction saved an incorrect return address in the `ra` register, which compromised function calls.

The issue stemmed from the synchronous reading of the BRAM blocks. The instruction memory provided data with a full clock cycle delay after the PC update. This misaligned the fetch and decode phases. Since the designed datapath is single cycle, the instruction and the RAM data must be available in the exact same clock cycle they are requested.

To restore correct timing, the memories were converted to asynchronous reads. The write operations remained synchronized to the clock. At the implementation level, this required instructing Vivado to map the VHDL arrays as distributed memory instead of Block RAM.

```
1 instr <= mem(to_integer(unsigned(addr(11 downto 2))));
```

Listing 9.1: Combinational read of the instruction memory.

9.3 Timing Analysis and Derived Clock

During the first complete implementation of the SoC, the static timing analysis revealed an issue. The critical path of the single cycle processor required a propagation time of approximately 15.4

ns. The 100 MHz system clock imposes a strict limit of 10 ns. Consequently, the circuit presented a Setup Violation with a Worst Negative Slack of -5.5 ns.

To resolve this timing violation, a hardware frequency divider was introduced. This allows the CPU to operate on a dedicated 25 MHz clock domain. The VGA controller remained on the native 100 MHz domain. This higher frequency is used to derive the 25 MHz pixel clock, which is necessary for the 60 Hz screen refresh rate.

```

1 process(clk, rst_int)
2 begin
3     if rst_int = '1' then
4         clk_cnt <= (others => '0');
5         cpu_clk <= '0';
6     elsif rising_edge(clk) then
7         clk_cnt <= clk_cnt + 1;
8         if clk_cnt = "01" or clk_cnt = "11" then
9             cpu_clk <= not cpu_clk; -- 100 MHz / 4 = 25 MHz
10        end if;
11    end if;
12 end process;

```

Listing 9.2: Clock divider to scale the CPU domain to 25 MHz.

With this addition, the timing analysis results were misleading. Vivado reported that the timing constraints were met. However, the static analysis actually excluded the CPU registers. This occurred because the synthesizer did not recognize the new clock domain. To resolve this, an explicit constraint was added to the XDC file to declare the derived clock:

```

1 create_generated_clock -name cpu_clk -source [get_ports clk] \
2   -divide_by 4 [get_pins cpu_clk_reg/Q]

```

Listing 9.3: Declaration of the derived clock in the constraints file.

Thanks to this directive, the Static Timing Analysis covered the entire project. The final implementation report certifies that both clock domains meet the timing constraints. It confirms that there are no unanalyzed endpoints and guarantees a positive WNS.

9.4 Bitstream Generation and Hardware Testing

After completing the implementation without errors or timing violations, the final bitstream was generated. This bitstream was used to program the board using the Vivado Hardware Manager.

Initially, a basic test firmware was compiled and loaded into the LUTRAM. This software only turned on the GPIO LEDs in an alternating pattern. It also moved the ball diagonally on the VGA monitor, completely ignoring the MEMS sensor data.

The success of this test verified that several components functioned correctly on the real hardware. These components included the CPU, clock divider, LUTRAM memories, GPIO, and VGA rendering. Consequently, this allowed the final debugging phases to focus exclusively on the SPI communication logic with the accelerometer.

Resource	Used	availability	Use
Slice LUTs	2529	63400	3,99%
LUT as Logic	2017	63400	3,18%
LUT as Memory	512	19000	2,69%
Slice Registers	1167	126800	0,92%
Block RAM Tile	0	135	0,00%
DSPs	13	240	5,42%
Bonded IOB	73	210	34,76%
BUFGCTRL	2	32	6,25%

Table 9.1: Artix-7 FPGA resource utilization after implementation.

Table 9.1 summarizes the FPGA’s physical resource utilization after implementation. This data confirms the architectural choices described in the previous chapters. The absence of BRAM blocks and the use of 512 LUTs in distributed memory mode prove that the memories are mapped to LUTRAM. Additionally, the 13 DSP blocks correspond to the inferred multiplications used to render the ball in the `vga_ball` module.

Chapter 10

The Game: Design and Final Testing

The final stage of the project involves creating a complete interactive video game. A ball is controlled by physically tilting the board, which uses data from the integrated accelerometer. The ball must navigate a maze and avoid traps to reach the finish line. This chapter describes the graphic design of the level, the collision rules, and the results of the final hardware testing.

10.1 The game elements

The ball is a red circle with a 10 pixel radius. To mathematically determine if a pixel belongs to the circle, the system uses the circle equation $(x - x_c)^2 + (y - y_c)^2 \leq r^2$. This method compares the squares of the distances. This choice avoids calculating square roots in the hardware. It also allows the synthesizer to map the multiplications directly onto the FPGA's native DSP blocks.

The maze walls are gray rectangles. Each wall has a dual identity in the system and shares the exact same coordinates. This identity exists at the hardware level in the VHDL `vga_ball` module for video signal generation, and at the software level in the C firmware for physics calculations. The collision logic compares the ball's bounding box (20 x 20 pixels) with the wall's bounding box. In the event of an impact or overlap, the velocity vector resets to zero. Consequently, the ball's position stops updating along that specific axis.

```
1 int wall_hit(int px, int py, int xmin, int xmax, int ymin, int ymax) {  
2     return (px + 20 > xmin && px < xmax &&  
3         py + 20 > ymin && py < ymax);  
4 }
```

Listing 10.1: C function for collision detection with maze walls.

The holes are black circles with a 25 pixel radius. If the center of the ball enters the tolerance area of a hole, a game over condition triggers. The firmware then stops the movement, turns on four GPIO LEDs, and resets the ball's coordinates to the starting point.

The finish line is a green rectangle. When the ball reaches it, the processor turns on all sixteen LEDs on the Nexys board to indicate a victory. It then returns the ball to the starting position. The LEDs turn off automatically when the board is physically tilted, which starts a new game.

10.2 The level design

The maze layout was designed using an iterative approach. Various versions were tested to balance the corridor widths, the hole placements, and the wall structures. The final version features thirteen walls and six holes.

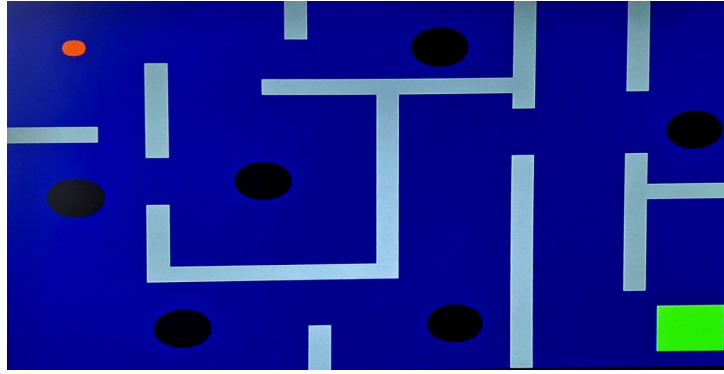


Figure 10.1: Complete maze.

10.3 Gaming on the hardware

The figures show the three main state conditions of the game logic. The LEDs remain off during normal movement or when the ball is stationary. When the ball falls into a hole, the CPU turns on four warning LEDs. Finally, when the ball reaches the finish line, the victory condition turns on all the LEDs on the board.

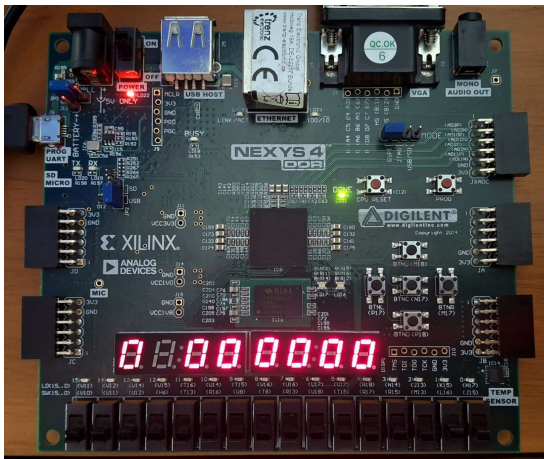


Figure 10.2: Resting state.

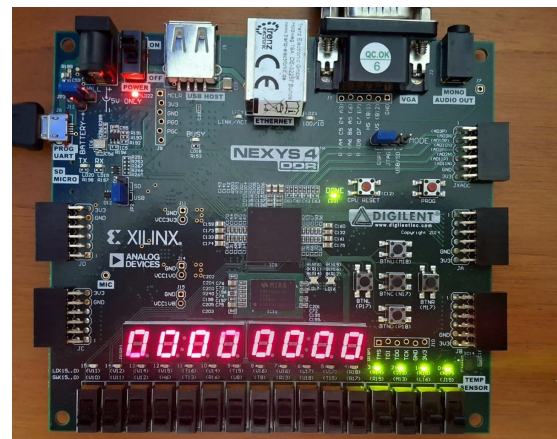


Figure 10.3: Fell into a hole.

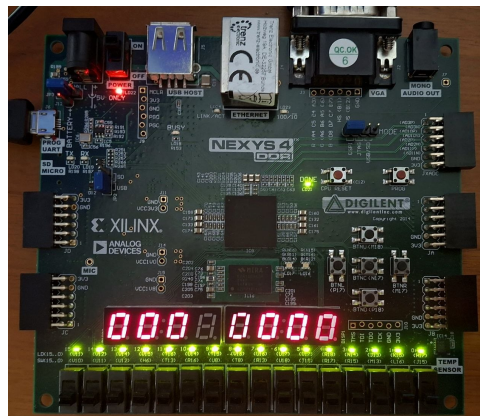


Figure 10.4: Victory.

Chapter 11

Conclusions

The System on Chip can autonomously execute the game firmware. It also communicates in real time with the accelerometer and the VGA monitor.

This project showed that moving from behavioral simulation to real hardware implementation is a distinct development phase. This phase features physical and timing issues that ideal models cannot reveal. The four most instructive cases were:

- **Single-Cycle Constraint:** In a non pipelined processor, the Instruction Memory and Data Memory cannot have latency cycles. They require combinational reading. Otherwise, a timing mismatch between the Program Counter update and the actual instruction fetch corrupts the datapath and the return addresses.
- **Timing Analysis and Clock Domains:** Timing constraints must declare all active clock domains. Without this step, the Static Timing Analysis ignores entire branches of the circuit and provides an incorrect performance estimate. Furthermore, resolving Setup Violations on the critical path required a divided 25 MHz clock dedicated strictly to the CPU.
- **Real Peripherals Timing:** Physical hardware devices require much stricter timing protocols than the dummy slaves used in testbenches.
- **Application Binary Interface (ABI):** Bare metal software must consider the architectural interface. Low level details directly affect physical calculations. This required explicit casting in the C firmware to ensure correct sign extension on the raw acceleration data.

The developed architecture supports future extensions. From a software perspective, it is possible to add multiple levels that can be selected using the hardware switches. Other software additions could include a life counter or a timer on the display, and a VGA framebuffer with more complex textures and colors.

On the hardware front, the processor could evolve into a 5 stage pipeline structure. This change would reduce the combinational critical path. It would also allow the CPU clock frequency to return to 100 MHz or higher, which would greatly increase the overall performance of the SoC.